

SQL SERVER 2025 QUERY STORE

End-to-End Technical SOP & Hands-On Training Guide

AdventureWorks2025 • Parameter Sniffing • PSP • Query Store Hints • Plan Forcing • Extended Events • OPPO

LEVEL	AUDIENCE	PURPOSE
Basic → Advanced	DBA / Engineer / Architect	Instructor-Led Training

Prepared for technical training purposes

Validated against Microsoft SQL Server documentation

Table of Contents

Use this index for fast classroom navigation. Page numbers reflect this published training edition.

Section	Coverage	Page
1. Purpose and Training Outcomes	Objectives, scope, learning outcomes	3
2. Lab Environment and Scope	SQL Server 2025, AdventureWorks2025, compatibility 170	3
3. Query Store Fundamentals	Architecture, stores, historical performance model	3
Steps 1-10	Environment, Query Store setup, skewed lab data, statistics	4
Steps 11-20	Stored procedure, classic parameter sniffing reproduction	7
Steps 21-30	Query Store history, metrics, waits, SSMS reports	8
Steps 31-40	Plan forcing, Query Store hints, rollback	11
Steps 41-50	RECOMPILE, PSP activation, dispatcher and variants	12
4. Advanced SQL Server 2025 PSP / XE / OPPO	Advanced optimization and diagnostics	14
Steps 51-70	PSP prerequisites, variants, histogram, Extended Events	14
Steps 71-80	Variant waits, plan forcing, hints, OPPO introduction	19
Steps 81-100	OPPO lab, production workflow, high variation, operating model	20
5. Fix Selection Matrix	PSP vs OPPO vs forcing vs hints vs recompile	26
6. Query Store vs Plan Cache	Operational comparison	26
7. DBA Troubleshooting Flowchart	Production investigation sequence	26
8. Training Instructor Notes	Delivery guidance for classroom sessions	27
9. Lab Cleanup / Reset	Stop XE sessions and optional object cleanup	27
10. Microsoft Official References	Microsoft Learn references only	28
Appendix A	Query Store DBA quick reference	29
Appendix B	Core concepts to remember	29

1. Purpose and Training Outcomes

This SOP is a complete instructor-ready lab for teaching SQL Server 2025 Query Store from fundamentals through advanced parameter-sensitive troubleshooting. It uses AdventureWorks2025 as the host database and creates an isolated QueryStoreLab workload so that parameter skew, plan reuse, Query Store history, PSP, Query Store hints, plan forcing, Extended Events diagnostics, and OPPO can be demonstrated repeatedly without altering core AdventureWorks tables.

- Understand what Query Store captures and why it is more useful than plan cache alone for historical performance analysis.
- Create a reproducible parameter-sensitive workload with a stored procedure that behaves differently for small and very large parameter populations.
- Diagnose plan reuse using actual execution plans, compiled/runtime values, statistics, Query Store runtime metrics, and Query Store waits.
- Apply and reverse plan forcing and Query Store hints without changing application source code.
- Use SQL Server 2025 Parameter Sensitive Plan optimization (PSP), dispatcher plans, query variants, and Extended Events diagnostics.
- Use SQL Server 2025 Optional Parameter Plan Optimization (OPPO) for optional NULL/non-NULL search predicates.
- Follow a production-safe troubleshooting workflow that preserves evidence and provides rollback paths.

Important — Parameter sniffing is normal optimizer behavior. The problem is parameter sensitivity: one cached plan is materially inappropriate for another part of the parameter distribution.

2. Lab Environment and Scope

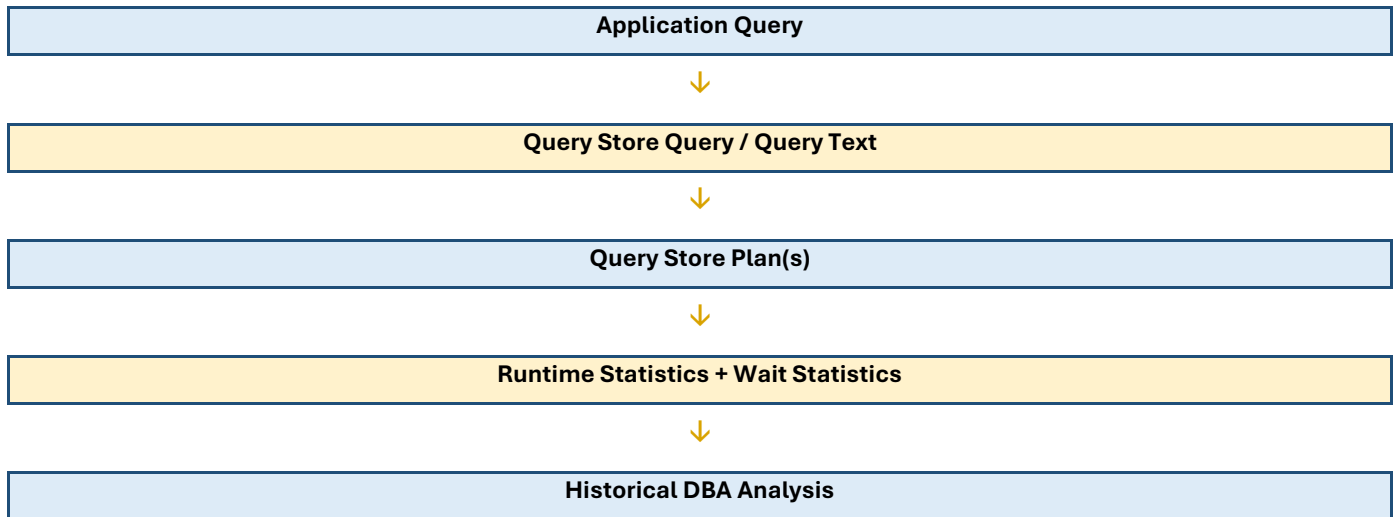
Item	Training Value	Notes
SQL Server	SQL Server 2025	Use a current SQL Server 2025 build for the lab.
Database	AdventureWorks2025	Microsoft sample database used as the host database.
Compatibility level	170	Required for the SQL Server 2025 optimizer environment and OPPO.
Lab schema	QueryStoreLab	Keeps training objects isolated.
Large parameter	CustomerID = 1	Approximately 500,000 rows.
Small parameters	CustomerID = 1001 / 5000	Approximately 10 rows per customer.
Client	SQL Server Management Studio	Enable Actual Execution Plan, STATISTICS IO, and STATISTICS TIME.

3. Query Store Fundamentals

Query Store acts as a database-level performance history repository. It stores query text, query identities, execution plans, runtime statistics, and query-level wait statistics. This makes it possible to answer not only “what is slow now?” but also “when did it become slow, which plan was active, and how did resource consumption change?”

Component	What it stores	Primary DBA use
Query text/query metadata	Normalized query text and query identity	Locate the statement and correlate it across plans.
Plan store	Historical execution plans and forcing state	Compare good/bad plans and force a known-good plan when justified.
Runtime statistics store	Executions, duration, CPU, logical I/O by interval	Trend performance over time and compare plans.
Wait statistics store	Wait categories associated with a Query Store plan	Separate CPU/I/O/locking/resource issues from plan-shape issues.

Query Store Architecture



Step 1 — Validate SQL Server 2025 and AdventureWorks2025

Confirm the SQL Server build and the database compatibility level before changing optimizer-related settings.

```

SELECT
    @@SERVERNAME AS ServerName,
    SERVERPROPERTY('ProductVersion') AS ProductVersion,
    SERVERPROPERTY('ProductLevel') AS ProductLevel,
    SERVERPROPERTY('Edition') AS Edition,
    @@VERSION AS FullVersion;
GO

SELECT name, compatibility_level, recovery_model_desc
FROM sys.databases
WHERE name = N'AdventureWorks2025';
GO

ALTER DATABASE AdventureWorks2025
SET COMPATIBILITY_LEVEL = 170;
GO
  
```

Step 2 — Enable Query Store BEFORE generating the workload

This is a mandatory lab prerequisite. Execute this step successfully before creating the parameter-sniffing workload. Query Store cannot capture historical query text, plans, runtime statistics, or waits while it is OFF. The 5-minute runtime-statistics interval is intentionally short for lab visibility; production settings must be sized for the workload and retention requirements.

```

USE master;
GO

ALTER DATABASE AdventureWorks2025
SET QUERY_STORE = ON
(
    OPERATION_MODE = READ_WRITE,
    QUERY_CAPTURE_MODE = AUTO,
    WAIT_STATS_CAPTURE_MODE = ON,
    INTERVAL_LENGTH_MINUTES = 5,
    MAX_STORAGE_SIZE_MB = 1024,
    SIZE_BASED_CLEANUP_MODE = AUTO
);
GO
  
```

Step 3 — Verify Query Store is READ_WRITE before continuing

```
USE AdventureWorks2025;
GO

SELECT
    actual_state_desc,
    desired_state_desc,
    query_capture_mode_desc,
    wait_stats_capture_mode_desc,
    current_storage_size_mb,
    max_storage_size_mb,
    readonly_reason,
    interval_length_minutes,
    size_based_cleanup_mode_desc
FROM sys.database_query_store_options;
GO
```

MANDATORY LAB GATE

Continue only when:

```
actual_state_desc      = READ_WRITE
desired_state_desc     = READ_WRITE
query_capture_mode_desc = AUTO (or ALL for a deliberate lab capture test)
wait_stats_capture_mode = ON
readonly_reason        = 0
```

If actual_state_desc = OFF and desired_state_desc = OFF, Query Store is disabled. Return to Step 2 and enable it before generating workload. If desired state is READ_WRITE but actual state is READ_ONLY, investigate readonly_reason and storage pressure before relying on Query Store evidence.

Step 4 — Create the isolated training schema

```
USE AdventureWorks2025;
GO
IF SCHEMA_ID(N'QueryStoreLab') IS NULL
    EXEC(N'CREATE SCHEMA QueryStoreLab AUTHORIZATION dbo;');
GO
```

Step 5 — Create a skew-friendly orders table

```
DROP TABLE IF EXISTS QueryStoreLab.CustomerOrders;
GO
CREATE TABLE QueryStoreLab.CustomerOrders
(
    SalesID          bigint IDENTITY(1,1) NOT NULL,
    CustomerID       int NOT NULL,
    OrderDate        date NOT NULL,
    OrderAmount      money NOT NULL,
    OrderStatus      char(1) NOT NULL,
    SalesPersonID    int NULL,
    Notes            char(200) NULL,
    CONSTRAINT PK_QueryStoreLab_CustomerOrders
        PRIMARY KEY CLUSTERED (SalesID)
);
GO
```

Step 6 — Load the high-volume customer

CustomerID 1 deliberately receives approximately 500,000 rows.

```
INSERT INTO QueryStoreLab.CustomerOrders
(CustomerID, OrderDate, OrderAmount, OrderStatus, SalesPersonID, Notes)
SELECT TOP (500000)
    1,
    DATEADD(DAY, -(ROW_NUMBER() OVER(ORDER BY (SELECT NULL))) % 1000), CONVERT(date, GETDATE())),
    100.00, 'C', 277, REPLICATE('A', 200)
FROM sys.all_objects AS a
```

```
CROSS JOIN sys.all_objects AS b;
GO
```

Step 7 — Load thousands of low-volume customers

The remaining rows are spread across many customer IDs, producing a strong histogram skew.

```
INSERT INTO QueryStoreLab.CustomerOrders
(CustomerID, OrderDate, OrderAmount, OrderStatus, SalesPersonID, Notes)
SELECT TOP (100000)
    1000 + (ROW_NUMBER() OVER(ORDER BY (SELECT NULL)) % 10000),
    DATEADD(DAY, -(ROW_NUMBER() OVER(ORDER BY (SELECT NULL)) % 1000), CONVERT(date, GETDATE())),
    50.00, 'C', 278, REPLICATE('B', 200)
FROM sys.all_objects AS a
CROSS JOIN sys.all_objects AS b;
GO
```

Step 8 — Validate the data skew

```
SELECT TOP (20) CustomerID, COUNT_BIG(*) AS RowCount
FROM QueryStoreLab.CustomerOrders
GROUP BY CustomerID
ORDER BY RowCount DESC;
GO
SELECT COUNT_BIG(*) AS TotalRows,
    COUNT(DISTINCT CustomerID) AS DistinctCustomers
FROM QueryStoreLab.CustomerOrders;
```

Step 9 — Create a narrow nonclustered index

Only CustomerID is indexed. Other selected columns remain outside the nonclustered index, so low-cardinality executions may use Key Lookup while a huge population may prefer a scan.

```
CREATE NONCLUSTERED INDEX IX_CustomerOrders_CustomerID
ON QueryStoreLab.CustomerOrders(CustomerID);
GO
UPDATE STATISTICS QueryStoreLab.CustomerOrders WITH FULLSCAN;
```

Step 10 — Inspect the histogram

The histogram proves whether SQL Server can see the skew that makes different plans reasonable.

- Review RANGE_HI_KEY and EQ_ROWS for CustomerID 1.
- Compare large EQ_ROWS for CustomerID 1 with the small customer populations.
- Remember that the optimizer and PSP derive decisions from statistics rather than DBA-defined thresholds.

```
DBCC SHOW_STATISTICS
('QueryStoreLab.CustomerOrders', 'IX_CustomerOrders_CustomerID')
WITH HISTOGRAM;
GO
```

Step 11 — Create the parameterized stored procedure

```
CREATE OR ALTER PROCEDURE QueryStoreLab.usp_GetCustomerOrders
    @CustomerID int
AS
BEGIN
    SET NOCOUNT ON;
    SELECT SalesID, CustomerID, OrderDate, OrderAmount,
        OrderStatus, SalesPersonID, Notes
    FROM QueryStoreLab.CustomerOrders
    WHERE CustomerID = @CustomerID;
END;
```

Step 12 — Understand the classic plan-reuse problem

A first execution for a low-volume customer can compile a seek-plus-lookup plan. Reuse of that same plan for CustomerID 1 can turn a few lookups into hundreds of thousands of lookups. The opposite compilation order can produce a broad scan plan that is wasteful for a tiny result set.

Step 13 — Temporarily disable PSP

PSP can automatically solve the lab problem, so disable it temporarily to reproduce classic single-plan parameter sensitivity.

```
ALTER DATABASE SCOPED CONFIGURATION
SET PARAMETER_SENSITIVE_PLAN_OPTIMIZATION = OFF;
GO
SELECT name,value,value_for_secondary
FROM sys.database_scoped_configurations
WHERE name=N'PARAMETER_SENSITIVE_PLAN_OPTIMIZATION';
```

Step 14 — Enable actual plan, IO, and time

In SSMS, enable Include Actual Execution Plan (Ctrl+M), then enable per-query IO and timing output.

```
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
GO
```

Step 15 — Force a fresh compilation safely

Prefer targeted recompilation in the lab. Avoid clearing the entire instance cache as a normal troubleshooting first step.

```
EXEC sys.sp_recompile N'QueryStoreLab.usp_GetCustomerOrders';
GO
```

Avoid as a first response — DBCC FREEPROCCACHE clears plan-cache state broadly and can destroy evidence or create a compilation storm. Use only with deliberate scope and justification.

Step 16 — Compile for the small parameter first

- Inspect the actual plan.
- Record Estimated Number of Rows and Actual Number of Rows.
- Record logical reads, CPU time, and elapsed time.
- Note whether the access path is Index Seek + Key Lookup.

```
EXEC QueryStoreLab.usp_GetCustomerOrders @CustomerID = 1001;
GO
```

Step 17 — Inspect compiled and runtime values

Open the SELECT operator properties or plan XML and locate parameter information. On the compiling execution you may see ParameterCompiledValue = 1001 and ParameterRuntimeValue = 1001.

Step 18 — Reuse the plan for the huge parameter

- Do not recompile before this execution.
- Look for the same cached plan shape.
- Compare ParameterCompiledValue with ParameterRuntimeValue.
- Measure the increase in reads, CPU, and duration.

```
EXEC QueryStoreLab.usp_GetCustomerOrders @CustomerID = 1;
```

Step 19 — Interpret the bad reuse correctly

A compiled value of 1001 and runtime value of 1 is evidence of plan reuse, not automatic proof of a defect. The performance regression, cardinality mismatch, and data skew together establish parameter sensitivity.

Step 20 — Reverse the compilation order

- The large-first compilation can produce a scan-oriented plan.
- Reuse for a tiny customer may then perform far more work than necessary.
- This demonstrates why one universally optimal plan may not exist.

```
EXEC sys.sp_recompile N'QueryStoreLab.usp_GetCustomerOrders';
GO
EXEC QueryStoreLab.usp_GetCustomerOrders @CustomerID = 1;
GO
EXEC QueryStoreLab.usp_GetCustomerOrders @CustomerID = 1001;
```

Compile value	Runtime value	Typical plan tendency	Interpretation
1001	1001	Seek + Lookup	Efficient for tiny population.
1001	1	Same seek reused	Potentially excessive lookups / high reads.
1	1	Broad scan / large-result strategy	Reasonable for huge population.
1	1001	Large-result plan reused	Potentially wasteful for tiny population.

Step 21 — Generate repeatable Query Store history

```
DECLARE @Counter int = 1;
WHILE @Counter <= 20
BEGIN
    EXEC QueryStoreLab.usp_GetCustomerOrders @CustomerID = 1001;
    SET @Counter += 1;
END;
GO

DECLARE @Counter2 int = 1;
WHILE @Counter2 <= 5
BEGIN
    EXEC QueryStoreLab.usp_GetCustomerOrders @CustomerID = 1;
    SET @Counter2 += 1;
END;
GO

DECLARE @Counter3 int = 1;
WHILE @Counter3 <= 20
BEGIN
    EXEC QueryStoreLab.usp_GetCustomerOrders @CustomerID = 5000;
    SET @Counter3 += 1;
END;
GO
```

Step 22 — Flush Query Store data for immediate lab visibility

```
EXEC sys.sp_query_store_flush_db;
GO
```

Step 23 — Confirm Query Store captured the procedure, then find query_id

```
USE AdventureWorks2025;
GO

/* 23A - Confirm the procedure resolves in the current database */
SELECT
    DB_NAME() AS CurrentDatabase,
    OBJECT_ID(N'QueryStoreLab.usp_GetCustomerOrders') AS ProcedureObjectID;
```

```

GO

/* 23B - Confirm Query Store is still available */
SELECT
    actual_state_desc,
    desired_state_desc,
    query_capture_mode_desc,
    wait_stats_capture_mode_desc,
    current_storage_size_mb,
    max_storage_size_mb,
    readonly_reason
FROM sys.database_query_store_options;
GO

/* 23C - Find the captured stored-procedure statement */
SELECT
    q.query_id,
    q.query_text_id,
    q.object_id,
    OBJECT_SCHEMA_NAME(q.object_id) AS SchemaName,
    OBJECT_NAME(q.object_id) AS ObjectName,
    qt.query_sql_text
FROM sys.query_store_query AS q
JOIN sys.query_store_query_text AS qt
    ON q.query_text_id = qt.query_text_id
WHERE q.object_id = OBJECT_ID(N'QueryStoreLab.usp_GetCustomerOrders')
ORDER BY q.query_id;
GO

```

Lab discipline — Always use the query_id returned by your own database. Do not reuse example IDs from training material.

Step 23D — If Step 23 still returns zero rows, diagnose capture before continuing

A successful SELECT with column headings but no rows means the metadata query itself worked; Query Store simply has no matching captured statement. Do not guess the query_id and do not continue to plan/runtime analysis until this is resolved.

```

/* Overall Query Store population */
SELECT COUNT(*) AS QueryStoreQueryCount
FROM sys.query_store_query;

SELECT COUNT(*) AS QueryStorePlanCount
FROM sys.query_store_plan;

SELECT COUNT(*) AS QueryStoreRuntimeStatsCount
FROM sys.query_store_runtime_stats;
GO

/* Broad fallback search by text */
SELECT
    q.query_id,
    q.query_text_id,
    q.object_id,
    OBJECT_SCHEMA_NAME(q.object_id) AS SchemaName,
    OBJECT_NAME(q.object_id) AS ObjectName,
    qt.query_sql_text
FROM sys.query_store_query AS q
JOIN sys.query_store_query_text AS qt
    ON q.query_text_id = qt.query_text_id
WHERE qt.query_sql_text LIKE N'%CustomerOrders%'
    OR qt.query_sql_text LIKE N'%CustomerID%'
ORDER BY q.query_id;
GO

```

Step 23E — Recovery procedure when Query Store was OFF

If Step 3 shows `actual_state_desc = OFF` and `desired_state_desc = OFF`, Query Store was disabled and none of the earlier workload was captured. Enable Query Store, verify `READ_WRITE`, then regenerate representative executions. Historical executions that occurred while Query Store was OFF cannot be reconstructed by Query Store.

```
USE master;
GO
ALTER DATABASE AdventureWorks2025
SET QUERY_STORE = ON
(
    OPERATION_MODE = READ_WRITE,
    QUERY_CAPTURE_MODE = AUTO,
    WAIT_STATS_CAPTURE_MODE = ON,
    INTERVAL_LENGTH_MINUTES = 5,
    MAX_STORAGE_SIZE_MB = 1024,
    SIZE_BASED_CLEANUP_MODE = AUTO
);
GO

USE AdventureWorks2025;
GO

/* Verify READ_WRITE before proceeding */
SELECT actual_state_desc, desired_state_desc,
       query_capture_mode_desc, wait_stats_capture_mode_desc,
       current_storage_size_mb, max_storage_size_mb, readonly_reason
FROM sys.database_query_store_options;
GO

/* Targeted fresh compilation and representative executions */
EXEC sys.sp_recompile N'QueryStoreLab.usp_GetCustomerOrders';
GO
EXEC QueryStoreLab.usp_GetCustomerOrders @CustomerID = 1001;
EXEC QueryStoreLab.usp_GetCustomerOrders @CustomerID = 1;
EXEC QueryStoreLab.usp_GetCustomerOrders @CustomerID = 5000;
EXEC QueryStoreLab.usp_GetCustomerOrders @CustomerID = 1001;
EXEC QueryStoreLab.usp_GetCustomerOrders @CustomerID = 1;
GO

EXEC sys.sp_query_store_flush_db;
GO
```

Step 23F — Lab-only capture-mode fallback

For this stored-procedure lab, `AUTO` is normally appropriate. If Query Store is `READ_WRITE` but a controlled test still does not appear, temporarily using `ALL` can remove capture-policy uncertainty while troubleshooting. Restore `AUTO` after the test. Do not make `ALL` a blanket production setting without evaluating workload volume and storage impact.

```
/* Optional controlled-lab diagnostic only */
ALTER DATABASE AdventureWorks2025
SET QUERY_STORE (QUERY_CAPTURE_MODE = ALL);
GO

EXEC sys.sp_recompile N'QueryStoreLab.usp_GetCustomerOrders';
EXEC QueryStoreLab.usp_GetCustomerOrders @CustomerID = 1001;
EXEC QueryStoreLab.usp_GetCustomerOrders @CustomerID = 1;
EXEC sys.sp_query_store_flush_db;
GO

/* Restore the intended lab setting */
ALTER DATABASE AdventureWorks2025
SET QUERY_STORE (QUERY_CAPTURE_MODE = AUTO);
GO
```

Expected result: Step 23 should return the statement inside QueryStoreLab.usp_GetCustomerOrders with a real query_id. Always use the query_id returned by your own database; example IDs in training material are not portable.

Step 24 — Capture the Query ID programmatically

```
DECLARE @QueryID bigint;

SELECT TOP (1)
    @QueryID = q.query_id
FROM sys.query_store_query AS q
WHERE q.object_id = OBJECT_ID(N'QueryStoreLab.usp_GetCustomerOrders')
ORDER BY q.query_id DESC;

SELECT
    @QueryID AS QueryStoreQueryID,
    CASE
        WHEN @QueryID IS NULL THEN 'STOP - Query Store has not captured the procedure statement.'
        ELSE 'OK - Continue with plan and runtime analysis.'
    END AS LabStatus;

GO
```

Step 25 — List plans captured for the procedure

```
SELECT q.query_id,p.plan_id,p.plan_type_desc,
    p.is_forced_plan,p.is_parallel_plan,p.count_compiles,
    p.last_compile_start_time,p.force_failure_count,
    p.last_force_failure_reason_desc
FROM sys.query_store_query AS q
JOIN sys.query_store_plan AS p ON q.query_id=p.query_id
WHERE q.object_id=OBJECT_ID(N'QueryStoreLab.usp_GetCustomerOrders')
ORDER BY q.query_id,p.plan_id;

GO
```

Step 26 — Calculate weighted runtime averages

Runtime statistics are aggregated by intervals, so weight interval averages by execution count rather than averaging averages.

```
SELECT q.query_id,p.plan_id,
    SUM(rs.count_executions) AS ExecutionCount,
    CAST(SUM(rs.avg_duration*rs.count_executions)
        /NULLIF(SUM(rs.count_executions),0)/1000.0 AS decimal(18,2)) AS WeightedAvgDuration_ms,
    CAST(SUM(rs.avg_cpu_time*rs.count_executions)
        /NULLIF(SUM(rs.count_executions),0)/1000.0 AS decimal(18,2)) AS WeightedAvgCPU_ms,
    CAST(SUM(rs.avg_logical_io_reads*rs.count_executions)
        /NULLIF(SUM(rs.count_executions),0) AS decimal(18,2)) AS WeightedAvgLogicalReads,
    MAX(rs.max_duration)/1000.0 AS MaxDuration_ms,
    MAX(rs.max_cpu_time)/1000.0 AS MaxCPU_ms
FROM sys.query_store_query AS q
JOIN sys.query_store_plan AS p ON q.query_id=p.query_id
JOIN sys.query_store_runtime_stats AS rs ON p.plan_id=rs.plan_id
WHERE q.object_id=OBJECT_ID(N'QueryStoreLab.usp_GetCustomerOrders')
GROUP BY q.query_id,p.plan_id
ORDER BY WeightedAvgDuration_ms DESC;

GO
```

Step 27 — Understand why weighted averages matter

An interval with 10,000 executions must contribute more than an interval with one execution. $\text{SUM}(\text{avg_value} \times \text{executions}) / \text{SUM}(\text{executions})$ better represents the workload than $\text{AVG}(\text{avg_value})$.

Step 28 — Trend runtime performance by interval

```
SELECT q.query_id,p.plan_id,rsi.start_time,rsi.end_time,
```

```

        rs.count_executions,
        CAST(rs.avg_duration/1000.0 AS decimal(18,2)) AS AvgDuration_ms,
        CAST(rs.min_duration/1000.0 AS decimal(18,2)) AS MinDuration_ms,
        CAST(rs.max_duration/1000.0 AS decimal(18,2)) AS MaxDuration_ms,
        CAST(rs.avg_cpu_time/1000.0 AS decimal(18,2)) AS AvgCPU_ms,
        rs.avg_logical_io_reads
FROM sys.query_store_query AS q
JOIN sys.query_store_plan AS p ON q.query_id=p.query_id
JOIN sys.query_store_runtime_stats AS rs ON p.plan_id=rs.plan_id
JOIN sys.query_store_runtime_stats_interval AS rsi
    ON rs.runtime_stats_interval_id=rsi.runtime_stats_interval_id
WHERE q.object_id=OBJECT_ID(N'QueryStoreLab.usp_GetCustomerOrders')
ORDER BY rsi.start_time DESC,p.plan_id;
GO

```

Step 29 — Inspect Query Store wait statistics

- Use wait categories to avoid misdiagnosing blocking, CPU pressure, I/O pressure, or memory/resource issues as parameter sniffing.
- Query Store waits are associated with Query Store plans, unlike broad instance wait accumulation.

```

SELECT q.query_id,p.plan_id,ws.wait_category_desc,
       SUM(ws.total_query_wait_time_ms) AS TotalWaitTime_ms
FROM sys.query_store_query AS q
JOIN sys.query_store_plan AS p ON q.query_id=p.query_id
JOIN sys.query_store_wait_stats AS ws ON p.plan_id=ws.plan_id
WHERE q.object_id=OBJECT_ID(N'QueryStoreLab.usp_GetCustomerOrders')
GROUP BY q.query_id,p.plan_id,ws.wait_category_desc
ORDER BY TotalWaitTime_ms DESC;
GO

```

Step 30 — Use SSMS Query Store reports

Navigate to AdventureWorks2025 > Query Store and review Regressed Queries, Overall Resource Consumption, Top Resource Consuming Queries, Queries With Forced Plans, Queries With High Variation, and Tracked Queries.

- Queries With High Variation is especially useful for “fast sometimes, slow sometimes” symptoms.
- High variation is a lead, not proof of parameter sensitivity.

Step 31 — Identify a known-good historical plan

Before forcing any plan, determine which parameters it handled well, why it was faster, whether its cardinality assumptions still match current data, and whether it is appropriate for the dominant production workload.

Step 32 — Force a known-good plan for emergency stabilization

```

EXEC sys.sp_query_store_force_plan
    @query_id = <YOUR_QUERY_ID>,
    @plan_id = <YOUR_GOOD_PLAN_ID>;
GO

```

Use case — Plan forcing is excellent for a verified plan regression when a previous plan is clearly better. It is not automatically the right permanent solution for a workload where different parameter ranges genuinely need different plans.

Step 33 — Verify forced-plan state and failures

```

SELECT query_id,plan_id,is_forced_plan,
       force_failure_count,last_force_failure_reason_desc
FROM sys.query_store_plan
WHERE query_id=<YOUR_QUERY_ID>;
GO

```

Step 34 — Retest both parameter extremes

Execute the small and huge customer with the forced plan and compare duration, CPU, logical reads, actual plan shape, and cardinality estimates. A plan that fixes one population but damages another is not a complete solution.

Step 35 — Rollback plan forcing

```
EXEC sys.sp_query_store_unforce_plan
    @query_id=<YOUR_QUERY_ID>,
    @plan_id=<YOUR_PLAN_ID>;
GO
```

Change control — Always prepare the unforce command before implementing production plan forcing.

Step 36 — Apply OPTIMIZE FOR UNKNOWN with a Query Store hint

This is a no-code-change test that can produce a more generic estimate-based plan.

```
EXEC sys.sp_query_store_set_hints
    @query_id=<YOUR_QUERY_ID>,
    @query_hints=N'OPTION (OPTIMIZE FOR UNKNOWN)';
GO
```

Step 37 — Inspect Query Store hint state

```
SELECT query_hint_id, query_id, query_hint_text, source_desc,
       query_hint_failure_count, last_query_hint_failure_reason_desc
FROM sys.query_store_query_hints
WHERE query_id=<YOUR_QUERY_ID>;
GO
```

Step 38 — Understand the UNKNOWN trade-off

OPTIMIZE FOR UNKNOWN can stabilize a workload by avoiding optimization for the currently sniffed value, but the generic plan may be mediocre for both extremes. Evaluate it with Query Store metrics, not assumptions.

Step 39 — Clear the Query Store hint

```
EXEC sys.sp_query_store_clear_hints
    @query_id=<YOUR_QUERY_ID>;
GO
```

Step 40 — Test DISABLE_PARAMETER_SNIFFING as a controlled hint

```
EXEC sys.sp_query_store_set_hints
    @query_id=<YOUR_QUERY_ID>,
    @query_hints=N'OPTION (USE HINT(''DISABLE_PARAMETER_SNIFFING''))';
GO
```

Interaction — Disabling parameter sniffing also prevents PSP from operating for that query. Use this only when you intentionally want that behavior.

Step 41 — Test RECOMPILE without changing procedure source

- Each execution can optimize for its current value.
- This can work well for infrequent, highly skewed procedures.
- Frequent execution can make compilation CPU significant.
- RECOMPILE prevents PSP from operating on that query.

```
EXEC sys.sp_query_store_clear_hints @query_id=<YOUR_QUERY_ID>;
GO
EXEC sys.sp_query_store_set_hints
```

```
@query_id=<YOUR_QUERY_ID>,
@query_hints=N'OPTION (RECOMPILE)';
GO
```

Step 42 — Clear RECOMPILE before PSP testing

```
EXEC sys.sp_query_store_clear_hints
@query_id=<YOUR_QUERY_ID>;
GO
```

Step 43 — Understand OPTIMIZE FOR a specific value

OPTION (OPTIMIZE FOR (@CustomerID = 1001)) can be useful only when a stable representative value is known. Data distributions change, so hard-coded assumptions require periodic review.

Step 44 — Re-enable SQL Server 2025 PSP

```
ALTER DATABASE SCOPED CONFIGURATION
SET PARAMETER_SENSITIVE_PLAN_OPTIMIZATION = ON;
GO
ALTER DATABASE AdventureWorks2025
SET COMPATIBILITY_LEVEL = 170;
GO
EXEC sys.sp_recompile N'QueryStoreLab.usp_GetCustomerOrders';
GO
```

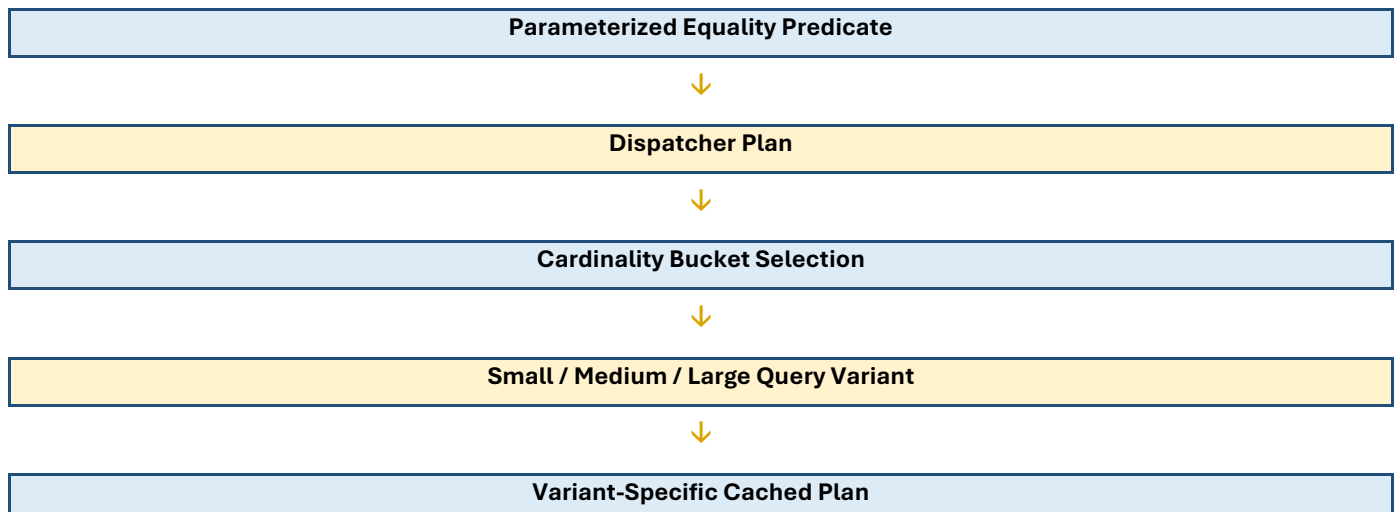
Step 45 — Generate multiple parameter populations for PSP

```
EXEC QueryStoreLab.usp_GetCustomerOrders @CustomerID=1001;
GO
EXEC QueryStoreLab.usp_GetCustomerOrders @CustomerID=1;
GO
EXEC QueryStoreLab.usp_GetCustomerOrders @CustomerID=5000;
GO
EXEC QueryStoreLab.usp_GetCustomerOrders @CustomerID=1;
GO
EXEC QueryStoreLab.usp_GetCustomerOrders @CustomerID=1001;
GO
```

Step 46 — Understand the PSP dispatcher model

When eligible, PSP keeps a dispatcher plan plus separate query variants. The dispatcher evaluates the runtime parameter against statistics-derived boundaries and routes execution to a variant optimized for that range.

PSP Routing Model



Step 47 — Inspect Query Store plan types

```
SELECT q.query_id,p.plan_id,p.plan_type,p.plan_type_desc,
       p.is_forced_plan,p.count_compiles,p.last_compile_start_time
FROM sys.query_store_query AS q
JOIN sys.query_store_plan AS p ON q.query_id=p.query_id
WHERE q.object_id=OBJECT_ID(N'QueryStoreLab.usp_GetCustomerOrders')
      OR p.plan_type_desc IN(N'Dispatcher Plan',N'Query Variant Plan')
ORDER BY q.query_id,p.plan_id;
GO
```

Step 48 — Inspect sys.query_store_query_variant

This SQL Server 2025-aware Query Store view exposes the relationship between parent queries, dispatcher plans, and child query variants.

```
SELECT *
FROM sys.query_store_query_variant;
GO
```

Step 49 — Inspect dispatcher and variant Showplan XML

- Search for ParameterSensitivePredicate.
- Observe internally generated PLAN PER VALUE metadata.
- Do not copy internal optimizer-generated hints into application SQL.

```
SELECT p.query_id,p.plan_id,p.plan_type_desc,
       TRY_CONVERT(xml,p.query_plan) AS QueryPlan
FROM sys.query_store_plan AS p
WHERE p.plan_type_desc IN(N'Dispatcher Plan',N'Query Variant Plan');
GO
```

Step 50 — Summarize the classic versus PSP behavior

Classic parameter sensitivity attempts to reuse one plan for all values. PSP can keep multiple active plans and route runtime values through a dispatcher. This is why SQL Server 2025 should be evaluated before immediately applying legacy workarounds.

4. Advanced SQL Server 2025 PSP, Extended Events, and OPPO

Step 51 — Reconfirm PSP prerequisites

- Compatibility level 160 or later is required for PSP; level 170 provides SQL Server 2025 optimizer behavior.
- The query must have an eligible parameter-sensitive equality predicate.
- RECOMPILE and disabled parameter sniffing prevent PSP operation.

```
SELECT name,compatibility_level
FROM sys.databases
WHERE name=DB_NAME();
GO
SELECT name,value,value_for_secondary
FROM sys.database_scoped_configurations
WHERE name=N'PARAMETER_SENSITIVE_PLAN_OPTIMIZATION';
GO
```

Step 52 — Ensure no Query Store hints are masking PSP

```
SELECT q.query_id,q.object_id,
       OBJECT_SCHEMA_NAME(q.object_id) AS SchemaName,
       OBJECT_NAME(q.object_id) AS ObjectName,
       qt.query_sql_text
FROM sys.query_store_query AS q
```

```
JOIN sys.query_store_query_text AS qt
  ON q.query_text_id=qt.query_text_id
WHERE q.object_id=OBJECT_ID(N'QueryStoreLab.usp_GetCustomerOrders');
GO

SELECT query_hint_id,query_id,query_hint_text,source_desc
FROM sys.query_store_query_hints
WHERE query_id=<YOUR_QUERY_ID>;
GO
```

Step 53 — Remove prior forced plans before controlled testing

If the test query has a forced plan, unforce it before investigating native PSP selection.

```
SELECT query_id,plan_id,plan_type_desc,is_forced_plan
FROM sys.query_store_plan
WHERE is_forced_plan=1;
GO
```

Step 54 — Trigger a clean PSP compilation

```
EXEC sys.sp_recompile N'QueryStoreLab.usp_GetCustomerOrders';
GO
EXEC QueryStoreLab.usp_GetCustomerOrders 1001;
GO
EXEC QueryStoreLab.usp_GetCustomerOrders 1;
GO
EXEC QueryStoreLab.usp_GetCustomerOrders 5000;
GO
```

Step 55 — Confirm whether PSP actually engaged

Do not infer PSP from the SQL Server version alone. Query Store should show dispatcher/query-variant evidence when the query qualifies.

```
SELECT q.query_id,p.plan_id,p.plan_type,p.plan_type_desc,
       p.is_forced_plan,p.count_compiles,p.last_compile_start_time
FROM sys.query_store_query AS q
JOIN sys.query_store_plan AS p ON q.query_id=p.query_id
WHERE q.object_id=OBJECT_ID(N'QueryStoreLab.usp_GetCustomerOrders')
      OR p.plan_type_desc IN(N'Dispatcher Plan',N'Query Variant Plan')
ORDER BY q.query_id,p.plan_id;
GO
```

Step 56 — Interpret the dispatcher

The dispatcher is not simply another “good plan.” It evaluates runtime parameter values and maps them to optimizer-defined cardinality ranges so the correct child variant can execute.

Step 57 — Read query-variant relationships explicitly

```
SELECT qv.query_variant_query_id,
       qv.parent_query_id,
       qv.dispatcher_plan_id,
       qv.query_variant_id
FROM sys.query_store_query_variant AS qv
ORDER BY qv.parent_query_id,qv.query_variant_id;
GO
```

Step 58 — List all variants for one parent query

```
DECLARE @ParentQueryID bigint=<YOUR_PARENT_QUERY_ID>;
SELECT qv.parent_query_id,qv.dispatcher_plan_id,
       qv.query_variant_id,qv.query_variant_query_id,
```

```

        p.plan_id,p.plan_type_desc,p.is_forced_plan
FROM sys.query_store_query_variant AS qv
LEFT JOIN sys.query_store_plan AS p
    ON qv.query_variant_query_id=p.query_id
WHERE qv.parent_query_id=@ParentQueryID
ORDER BY qv.query_variant_id,p.plan_id;
GO

```

Step 59 — Avoid object_id confusion with variants

PSP variants execute as prepared statements. Their object association in traditional plan-cache DMVs can differ from the stored-procedure parent. Use sys.query_store_query_variant to map parent and child relationships.

Step 60 — Review parent and child Showplan together

```

DECLARE @ParentQueryID bigint=<YOUR_PARENT_QUERY_ID>;
SELECT p.query_id,p.plan_id,p.plan_type_desc,
    TRY_CONVERT(xml,p.query_plan) AS QueryPlan
FROM sys.query_store_plan AS p
WHERE p.query_id=@ParentQueryID
    OR p.query_id IN
    (SELECT query_variant_query_id
    FROM sys.query_store_query_variant
    WHERE parent_query_id=@ParentQueryID)
ORDER BY p.query_id,p.plan_id;
GO

```

Step 61 — Understand dispatcher boundaries

Boundary values are statistics-derived. They are not thresholds configured manually by a DBA. Changes to the histogram can therefore change how SQL Server buckets parameter values.

Step 62 — Reinspect statistics when PSP behavior changes

- Check whether the heavy value still has disproportionately high EQ_ROWS.
- Confirm statistics freshness before blaming PSP for unexpected routing or plan choice.

```

DBCC SHOW_STATISTICS
('QueryStoreLab.CustomerOrders','IX_CustomerOrders_CustomerID')
WITH HISTOGRAM;
GO

```

Step 63 — Identify PSP diagnostic Extended Events

Two important events are parameter_sensitive_plan_optimization_skipped_reason and query_with_parameter_sensitivity. Use them to replace guesswork with engine diagnostics.

Step 64 — List current PSP skip reasons

Querying the engine is preferable to hard-coding a skip-reason list because diagnostic values can evolve across releases.

```

SELECT map_key,map_value
FROM sys.dm_xe_map_values
WHERE name=N'psp_skipped_reason_enum'
ORDER BY map_key;
GO

```

Step 65 — Create the PSP diagnostic Extended Events session

```

IF EXISTS(SELECT 1 FROM sys.server_event_sessions WHERE name=N'PSP_Diagnostics')
    DROP EVENT SESSION PSP_Diagnostics ON SERVER;
GO

```

```

CREATE EVENT SESSION PSP_Diagnostics ON SERVER
ADD EVENT sqlserver.parameter_sensitive_plan_optimization_skipped_reason
(
    ACTION(sqlserver.database_name,sqlserver.sql_text,
           sqlserver.query_hash,sqlserver.session_id)
),
ADD EVENT sqlserver.query_with_parameter_sensitivity
(
    ACTION(sqlserver.database_name,sqlserver.sql_text,
           sqlserver.query_hash,sqlserver.session_id)
)
ADD TARGET package0.ring_buffer;
GO
ALTER EVENT SESSION PSP_Diagnostics ON SERVER STATE=START;
GO

```

Step 66 — Generate executions while PSP diagnostics are active

```

EXEC sys.sp_recompile N'QueryStoreLab.usp_GetCustomerOrders';
GO
EXEC QueryStoreLab.usp_GetCustomerOrders 1001;
GO
EXEC QueryStoreLab.usp_GetCustomerOrders 1;
GO
EXEC QueryStoreLab.usp_GetCustomerOrders 5000;
GO

```

Step 67 — Read the PSP Extended Events ring buffer

- Look for query_with_parameter_sensitivity when predicates are considered interesting.
- Look for parameter_sensitive_plan_optimization_skipped_reason when PSP does not engage.
- SQL Server 2025 diagnostics can expose richer predicate-level details.

```

SELECT CAST(target_data AS xml) AS XEData
FROM sys.dm_xe_session_targets AS xt
JOIN sys.dm_xe_sessions AS xs
    ON xt.event_session_address=xs.address
WHERE xs.name=N'PSP_Diagnostics'
    AND xt.target_name=N'ring_buffer';
GO

```

Step 68 — Test multiple parameter-sensitive predicates

SQL Server 2025 improves PSP handling when multiple eligible predicates exist. Still, SQL Server limits how many predicates are considered to control variant explosion and plan-cache growth.

```

CREATE OR ALTER PROCEDURE QueryStoreLab.usp_SearchAdvanced
    @CustomerID int,
    @SalesPersonID int
AS
BEGIN
    SELECT SalesID, CustomerID, SalesPersonID, OrderDate, OrderAmount
    FROM QueryStoreLab.CustomerOrders
    WHERE CustomerID=@CustomerID
        AND SalesPersonID=@SalesPersonID;
END;
GO

```

Step 69 — Measure runtime statistics for individual variants

```

DECLARE @ParentQueryID bigint=<YOUR_PARENT_QUERY_ID>;
SELECT qv.parent_query_id,qv.query_variant_id,
       qv.query_variant_query_id,p.plan_id,
       SUM(rs.count_executions) AS ExecutionCount,
       CAST(SUM(rs.avg_duration*rs.count_executions)

```

```

        /NULLIF(SUM(rs.count_executions),0)/1000.0 AS decimal(18,2)) AS WeightedAvgDuration_ms,
        CAST(SUM(rs.avg_cpu_time*rs.count_executions)
        /NULLIF(SUM(rs.count_executions),0)/1000.0 AS decimal(18,2)) AS WeightedAvgCPU_ms,
        CAST(SUM(rs.avg_logical_io_reads*rs.count_executions)
        /NULLIF(SUM(rs.count_executions),0) AS decimal(18,2)) AS WeightedAvgLogicalReads
FROM sys.query_store_query_variant AS qv
JOIN sys.query_store_plan AS p
    ON qv.query_variant_query_id=p.query_id
JOIN sys.query_store_runtime_stats AS rs
    ON p.plan_id=rs.plan_id
WHERE qv.parent_query_id=@ParentQueryID
GROUP BY qv.parent_query_id,qv.query_variant_id,
        qv.query_variant_query_id,p.plan_id
ORDER BY qv.query_variant_id,p.plan_id;
GO

```

Step 70 — Compare efficiency in context

A large-result variant naturally consumes more resources than a 10-row variant. Judge it by rows processed, estimated-versus-actual cardinality, logical reads, CPU, elapsed time, execution frequency, and waits rather than by one absolute metric.

Step 71 — Measure Query Store waits per variant

```

DECLARE @ParentQueryID bigint=<YOUR_PARENT_QUERY_ID>;
SELECT qv.query_variant_id,qv.query_variant_query_id,
        p.plan_id,ws.wait_category_desc,
        SUM(ws.total_query_wait_time_ms) AS TotalQueryWaitTime_ms
FROM sys.query_store_query_variant AS qv
JOIN sys.query_store_plan AS p
    ON qv.query_variant_query_id=p.query_id
JOIN sys.query_store_wait_stats AS ws
    ON p.plan_id=ws.plan_id
WHERE qv.parent_query_id=@ParentQueryID
GROUP BY qv.query_variant_id,qv.query_variant_query_id,
        p.plan_id,ws.wait_category_desc
ORDER BY TotalQueryWaitTime_ms DESC;
GO

```

Step 72 — Understand plan forcing with PSP

The normal sys.sp_query_store_force_plan and sys.sp_query_store_unforce_plan procedures apply to dispatcher and variant plans, but forcing a child variant is not the same as forcing its parent dispatcher.

Step 73 — Force a child variant plan when justified

This forces that child query plan. It does not automatically force the parent dispatcher.

```

EXEC sys.sp_query_store_force_plan
    @query_id=<VARIANT_QUERY_ID>,
    @plan_id=<VARIANT_PLAN_ID>;
GO

```

Step 74 — Force a dispatcher deliberately

A forced dispatcher continues routing to its eligible variants. Treat forcing state carefully across dispatcher generations and always document rollback.

```

EXEC sys.sp_query_store_force_plan
    @query_id=<PARENT_QUERY_ID>,
    @plan_id=<DISPATCHER_PLAN_ID>;
GO

```

Step 75 — Apply a Query Store hint to the PSP parent

A parent Query Store hint can be inherited by child variants when they do not have their own more specific hint.

```
EXEC sys.sp_query_store_set_hints
    @query_id=<PARENT_QUERY_ID>,
    @query_hints=N'OPTION (MAXDOP 4)';
GO
```

Step 76 — Understand child hint precedence

A hint applied directly to a child variant can override the inherited parent hint for that child. This gives granular control but also increases operational complexity and should be documented carefully.

Step 77 — Recognize the RECOMPILE interaction with PSP

```
EXEC sys.sp_query_store_set_hints
    @query_id=<PARENT_QUERY_ID>,
    @query_hints=N'OPTION (RECOMPILE)';
GO
```

Critical — If the operational goal is to benefit from PSP, adding RECOMPILE contradicts that goal because PSP does not operate on a query carrying RECOMPILE.

Step 78 — Monitor Query Store hint failures

```
SELECT query_hint_id,query_id,query_hint_text,source_desc,
       query_hint_failure_count,last_query_hint_failure_reason_desc
FROM sys.query_store_query_hints;
GO
```

Step 79 — Introduce SQL Server 2025 OPPO

Optional Parameter Plan Optimization targets optional search patterns such as CustomerID = @CustomerID OR @CustomerID IS NULL. The NULL case can require a very broad result while the supplied-value case can be highly selective.

Step 80 — Verify OPPO configuration and compatibility

- OPPO is a SQL Server 2025 feature.
- Compatibility level 170 is required.
- OPPO uses the Multiplan infrastructure also used by PSP.

```
SELECT name,value
FROM sys.database_scoped_configurations
WHERE name=N'OPTIONAL_PARAMETER_OPTIMIZATION';
GO
SELECT name,compatibility_level
FROM sys.databases
WHERE name=N'AdventureWorks2025';
GO
```

Step 81 — Create the optional-search procedure

```
CREATE OR ALTER PROCEDURE QueryStoreLab.usp_SearchCustomerOrders
    @CustomerID int = NULL
AS
BEGIN
    SET NOCOUNT ON;
    SELECT SalesID,CustomerID,OrderDate,OrderAmount,
           OrderStatus,SalesPersonID,Notes
    FROM QueryStoreLab.CustomerOrders
    WHERE CustomerID=@CustomerID
```

```

OR @CustomerID IS NULL;
END;
GO

```

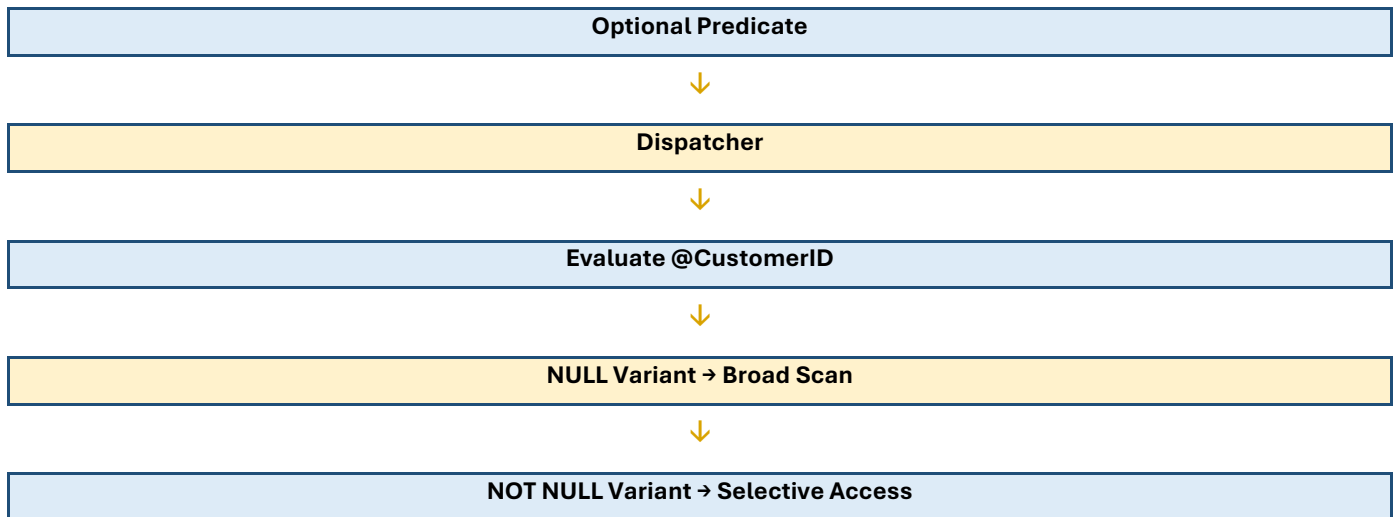
Step 82 — Understand the traditional optional-predicate dilemma

@CustomerID = 1001 returns only a few rows, while @CustomerID = NULL returns the entire table. A single reusable plan must be semantically safe for both cases and may be inefficient for one of them.

Step 83 — Understand OPPO routing

OPPO can create a dispatcher that separates NULL from NOT NULL behavior and routes each execution to a query variant specialized for that condition.

OPPO Routing Model



Step 84 — Generate an OPPO workload

```

EXEC sys.sp_recompile N'QueryStoreLab.usp_SearchCustomerOrders';
GO
EXEC QueryStoreLab.usp_SearchCustomerOrders @CustomerID=1001;
GO
EXEC QueryStoreLab.usp_SearchCustomerOrders @CustomerID=NULL;
GO
EXEC QueryStoreLab.usp_SearchCustomerOrders @CustomerID=5000;
GO
EXEC QueryStoreLab.usp_SearchCustomerOrders @CustomerID=NULL;
GO

```

Step 85 — Locate the OPPO parent query in Query Store

```

SELECT q.query_id,q.object_id,
       OBJECT_SCHEMA_NAME(q.object_id) AS SchemaName,
       OBJECT_NAME(q.object_id) AS ObjectName,
       qt.query_sql_text
FROM sys.query_store_query AS q
JOIN sys.query_store_query_text AS qt
  ON q.query_text_id=qt.query_text_id
WHERE q.object_id=OBJECT_ID(N'QueryStoreLab.usp_SearchCustomerOrders');
GO

```

Step 86 — List OPPO variants

```

DECLARE @ParentQueryID bigint=<YOUR_QUERY_ID>;
SELECT qv.parent_query_id,qv.dispatcher_plan_id,

```

```

        qv.query_variant_id,qv.query_variant_query_id,
        p.plan_id,p.plan_type_desc
FROM sys.query_store_query_variant AS qv
LEFT JOIN sys.query_store_plan AS p
    ON qv.query_variant_query_id=p.query_id
WHERE qv.parent_query_id=@ParentQueryID
ORDER BY qv.query_variant_id;
GO

```

Step 87 — Identify OPPO diagnostic events

Use `optional_parameter_optimization_skipped_reason` and `query_with_optional_parameter_predicate` to determine whether the pattern was recognized and why OPPO was or was not used.

Step 88 — List OPPO skip reasons and create diagnostics

```

SELECT map_key,map_value
FROM sys.dm_xe_map_values
WHERE name=N'opo_skipped_reason_enum'
ORDER BY map_key;
GO

IF EXISTS(SELECT 1 FROM sys.server_event_sessions WHERE name=N'OPPO_Diagnostics')
    DROP EVENT SESSION OPPO_Diagnostics ON SERVER;
GO
CREATE EVENT SESSION OPPO_Diagnostics ON SERVER
ADD EVENT sqlserver.optional_parameter_optimization_skipped_reason
(
    ACTION(sqlserver.database_name,sqlserver.sql_text,
        sqlserver.query_hash,sqlserver.session_id)
),
ADD EVENT sqlserver.query_with_optional_parameter_predicate
(
    ACTION(sqlserver.database_name,sqlserver.sql_text,
        sqlserver.query_hash,sqlserver.session_id)
)
ADD TARGET package0.ring_buffer;
GO
ALTER EVENT SESSION OPPO_Diagnostics ON SERVER STATE=START;
GO

```

Step 89 — Read OPPO Extended Events data

```

EXEC sys.sp_recompile N'QueryStoreLab.usp_SearchCustomerOrders';
GO
EXEC QueryStoreLab.usp_SearchCustomerOrders 1001;
GO
EXEC QueryStoreLab.usp_SearchCustomerOrders NULL;
GO

SELECT CAST(target_data AS xml) AS XEData
FROM sys.dm_xe_session_targets AS xt
JOIN sys.dm_xe_sessions AS xs
    ON xt.event_session_address=xs.address
WHERE xs.name=N'OPPO_Diagnostics'
    AND xt.target_name=N'ring_buffer';
GO

```

Step 90 — Disable OPPO for controlled before/after testing

```

ALTER DATABASE SCOPED CONFIGURATION
SET OPTIONAL_PARAMETER_OPTIMIZATION = OFF;
GO
EXEC sys.sp_recompile N'QueryStoreLab.usp_SearchCustomerOrders';
GO
SET STATISTICS IO ON;

```

```
SET STATISTICS TIME ON;
GO
EXEC QueryStoreLab.usp_SearchCustomerOrders 1001;
GO
EXEC QueryStoreLab.usp_SearchCustomerOrders NULL;
GO
```

Step 91 — Re-enable OPPO and compare metrics

- Compare plans, logical reads, CPU, duration, estimated rows, and actual rows.
- The goal is measured improvement, not simply proving that multiple plans exist.

```
ALTER DATABASE SCOPED CONFIGURATION
SET OPTIONAL_PARAMETER_OPTIMIZATION = ON;
GO
EXEC sys.sp_recompile N'QueryStoreLab.usp_SearchCustomerOrders';
GO
EXEC QueryStoreLab.usp_SearchCustomerOrders 1001;
GO
EXEC QueryStoreLab.usp_SearchCustomerOrders NULL;
GO
```

Step 92 — Disable OPPO for one query using Query Store hints

This query-scoped control is preferable to disabling OPPO database-wide when only one statement needs special treatment.

```
EXEC sys.sp_query_store_set_hints
    @query_id=<YOUR_PARENT_QUERY_ID>,
    @query_hints=N'OPTION (USE HINT(''DISABLE_OPTIONAL_PARAMETER_OPTIMIZATION''))';
GO

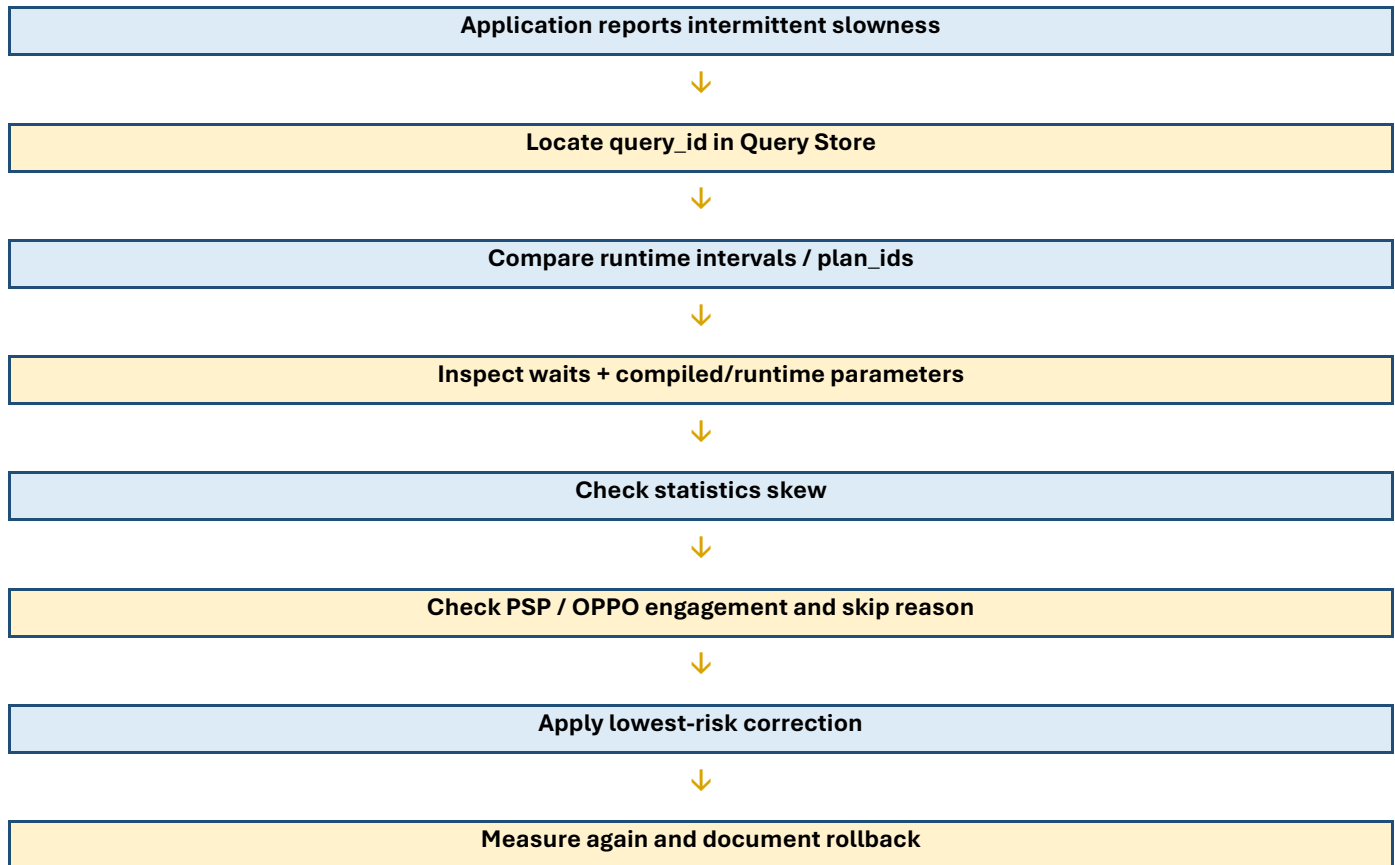
EXEC sys.sp_query_store_clear_hints
    @query_id=<YOUR_PARENT_QUERY_ID>;
GO
```

Step 93 — Distinguish PSP from OPPO

Area	PSP	OPPO
Problem shape	Different non-NULL parameter values need different plans	NULL versus non-NULL optional search needs different plans
Typical predicate	CustomerID = @CustomerID	CustomerID = @CustomerID OR @CustomerID IS NULL
Dispatcher / variants	Yes	Yes
SQL Server 2025	Enhanced	Introduced
Compatibility 170	Recommended for SQL Server 2025 behavior	Required
Query Store integration	Yes	Yes
Extended Events skip diagnostics	Yes	Yes

Step 94 — Follow a production incident workflow

For an intermittent procedure slowdown, preserve evidence first. Use Query Store to locate the query, compare time intervals and plans, inspect waits, examine parameter populations and statistics, then use PSP/OPPO diagnostics before applying a corrective action.

Production Investigation Flow**Step 95 — Find top Query Store plans by duration and CPU**

```

SELECT TOP (20)
    q.query_id,p.plan_id,qt.query_sql_text,
    SUM(rs.count_executions) AS ExecutionCount,
    CAST(SUM(rs.avg_duration*rs.count_executions)
        /NULLIF(SUM(rs.count_executions),0)/1000.0 AS decimal(18,2)) AS WeightedAvgDuration_ms,
    CAST(MAX(rs.max_duration)/1000.0 AS decimal(18,2)) AS MaxDuration_ms,
    CAST(SUM(rs.avg_cpu_time*rs.count_executions)
        /NULLIF(SUM(rs.count_executions),0)/1000.0 AS decimal(18,2)) AS WeightedAvgCPU_ms
FROM sys.query_store_query_text AS qt
JOIN sys.query_store_query AS q ON qt.query_text_id=q.query_text_id
JOIN sys.query_store_plan AS p ON q.query_id=p.query_id
JOIN sys.query_store_runtime_stats AS rs ON p.plan_id=rs.plan_id
GROUP BY q.query_id,p.plan_id,qt.query_sql_text
ORDER BY MaxDuration_ms DESC;
GO
  
```

Step 96 — Find high-variation Query Store candidates

```

SELECT TOP (50)
    q.query_id,p.plan_id,qt.query_sql_text,
    SUM(rs.count_executions) AS ExecutionCount,
    CAST(MIN(rs.min_duration)/1000.0 AS decimal(18,2)) AS MinimumDuration_ms,
    CAST(MAX(rs.max_duration)/1000.0 AS decimal(18,2)) AS MaximumDuration_ms,
    CAST(MAX(rs.max_duration)
        /NULLIF(MIN(NULLIF(rs.min_duration,0)),0) AS decimal(18,2)) AS DurationVariationRatio
FROM sys.query_store_query_text AS qt
JOIN sys.query_store_query AS q ON qt.query_text_id=q.query_text_id
JOIN sys.query_store_plan AS p ON q.query_id=p.query_id
JOIN sys.query_store_runtime_stats AS rs ON p.plan_id=rs.plan_id
GROUP BY q.query_id,p.plan_id,qt.query_sql_text
HAVING SUM(rs.count_executions)>=5
  
```

```
ORDER BY DurationVariationRatio DESC;
GO
```

Interpretation — High variation can come from blocking, I/O, CPU contention, spills, data volume, network/client behavior, or parameter sensitivity. Treat it as an investigation queue, not a diagnosis.

Step 97 — Use the production diagnosis checklist

- Confirm the statement is parameterized.
- Verify meaningful data skew in statistics.
- Prove material performance variation.
- Review Query Store plans/variants and actual execution plans.
- Compare estimated and actual cardinality.
- Check PSP or OPPO prerequisites and Extended Events skip reasons.
- Review Query Store waits for unrelated resource bottlenecks.
- Check existing forced plans and Query Store hints.
- Apply the lowest-risk targeted correction.
- Measure performance after the change and keep a rollback path.

Step 98 — Separate emergency stabilization from permanent remediation

A verified known-good plan can be forced quickly during an incident, but permanent remediation may involve statistics maintenance, indexing, query design, PSP/OPPO eligibility, compatibility settings, or workload/data-distribution changes. Query Store tells you whether the emergency action actually improved the workload.

Step 99 — Avoid common anti-patterns

- Do not clear the entire plan cache as the first troubleshooting step.
- Do not add RECOMPILE to every parameter-sensitive procedure.
- Do not use OPTIMIZE FOR UNKNOWN without comparing real workload metrics.
- Do not disable parameter sniffing merely because compiled and runtime values differ.
- Do not force one plan when different parameter populations legitimately need different strategies.
- Do not judge a large-result variant by raw CPU or reads without considering rows processed and execution frequency.

Step 100 — Apply the complete SQL Server 2025 Query Store operating model

Query Store supplies historical evidence; PSP handles value-based equality parameter sensitivity; OPPO handles optional NULL/non-NULL predicates; Extended Events explains engagement or skip reasons; Query Store hints and plan forcing provide targeted operational control; and measured before/after validation confirms whether the chosen fix is correct.

5. Fix Selection Matrix

Technique	Best fit	Strength	Primary caution
PSP	Eligible value-based parameter sensitivity	Multiple active plans automatically	Requires eligibility; do not mask it with incompatible hints.
OPPO	Optional NULL/non-NULL search predicates	Separate plans for optional-search semantics	SQL Server 2025 + compatibility 170.
Plan forcing	Verified regression with known-good historical plan	Fast emergency stabilization	One plan may not suit all parameter populations.
Query Store hint	Need no-code-change targeted behavior	Fast, persistent, reversible	Must be reviewed as data/workload changes.
RECOMPILE	Infrequent, highly skewed	Optimizes for current execution	Compilation CPU; PSP does not

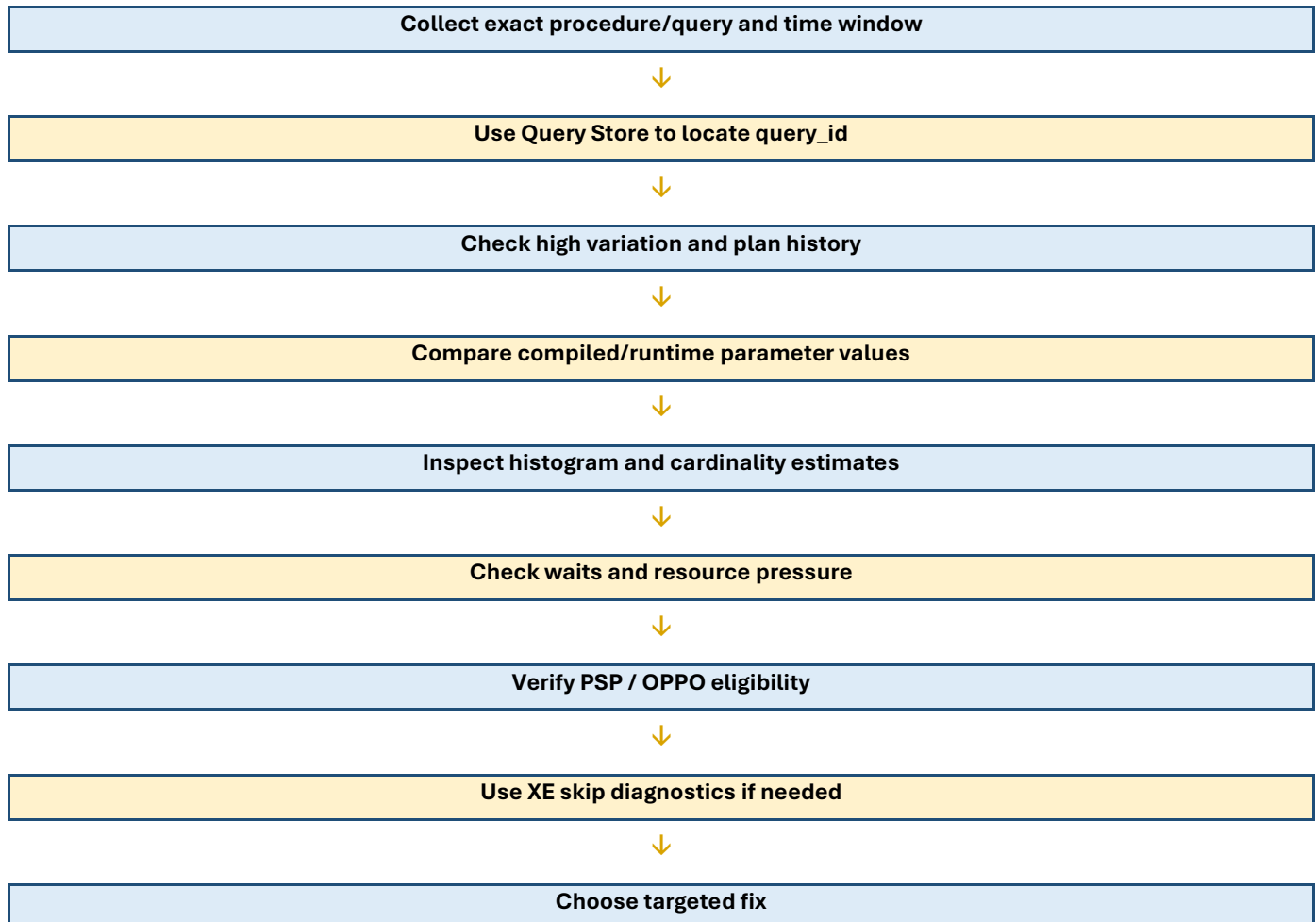
Technique	Best fit	Strength	Primary caution
	procedure		operate.
OPTIMIZE FOR UNKNOWN	Generic compromise plan is acceptable	Reduces value-specific sniffing	Can be mediocre for extremes.
OPTIMIZE FOR value	Stable representative parameter exists	Predictable chosen estimate	Representative value may become obsolete.
Index/query rewrite	Structural design issue	Can permanently reduce work	Requires application/change-management testing.

6. Query Store vs Plan Cache

Capability	Query Store	Plan Cache
Historical persistence	Yes, database-scoped persistence	No; entries can be evicted/cleared
Multiple historical plans	Yes	Only currently cached state
Runtime history by intervals	Yes	No comparable persisted interval history
Query-level wait history	Yes	No persisted Query Store-style history
Plan forcing	Yes	No Query Store forcing semantics
Query Store hints	Yes	Not a plan-cache feature
PSP/variant analysis	Strong historical visibility	Useful current cache detail, but less persistent
Best question	What changed over time?	What is cached now?

7. DBA Troubleshooting Flowchart

Intermittently Slow Stored Procedure





Re-run workload and compare Query Store metrics

8. Training Instructor Notes

- Run the classic parameter-sensitivity section with PSP OFF first. This makes the problem visible before demonstrating the SQL Server 2025 solution.
- Use sp_recompile rather than clearing the entire instance plan cache during demonstrations.
- Capture screenshots of the small-value plan, bad large-value reuse, Query Store runtime chart, dispatcher plan, and query variants if preparing classroom slides.
- Ask trainees to predict the likely plan before execution, then verify with actual plan and Query Store metrics.
- When a fix is applied, always test both the tiny and huge parameter populations. A change is incomplete if it improves only one side.
- Demonstrate rollback immediately after every plan force or Query Store hint so operational reversibility becomes part of the habit.

9. Lab Cleanup / Reset

Optional classroom reset

```
USE AdventureWorks2025;
GO
-- Stop diagnostic sessions if they exist
IF EXISTS(SELECT 1 FROM sys.server_event_sessions WHERE name=N'PSP_Diagnostics')
BEGIN
    ALTER EVENT SESSION PSP_Diagnostics ON SERVER STATE=STOP;
    DROP EVENT SESSION PSP_Diagnostics ON SERVER;
END;
GO
IF EXISTS(SELECT 1 FROM sys.server_event_sessions WHERE name=N'OPPO_Diagnostics')
BEGIN
    ALTER EVENT SESSION OPPO_Diagnostics ON SERVER STATE=STOP;
    DROP EVENT SESSION OPPO_Diagnostics ON SERVER;
END;
GO

-- Restore database-scoped optimizer settings
ALTER DATABASE SCOPED CONFIGURATION
SET PARAMETER_SENSITIVE_PLAN_OPTIMIZATION = ON;
GO
ALTER DATABASE SCOPED CONFIGURATION
SET OPTIONAL_PARAMETER_OPTIMIZATION = ON;
GO

-- Optional: remove training objects
DROP PROCEDURE IF EXISTS QueryStoreLab.usp_SearchAdvanced;
DROP PROCEDURE IF EXISTS QueryStoreLab.usp_SearchCustomerOrders;
DROP PROCEDURE IF EXISTS QueryStoreLab.usp_GetCustomerOrders;
DROP TABLE IF EXISTS QueryStoreLab.CustomerOrders;
GO
```

Caution — Do not run cleanup blindly in a shared training database if other students are still using the QueryStoreLab objects.

10. Microsoft Official References

1. Monitor performance by using Query Store

<https://learn.microsoft.com/en-us/sql/relational-databases/performance/monitoring-performance-by-using-the-query-store>

2. Tune performance with Query Store

<https://learn.microsoft.com/en-us/sql/relational-databases/performance/tune-performance-with-the-query-store?view=sql-server-ver17>

3. Query Store usage scenarios

<https://learn.microsoft.com/en-us/sql/relational-databases/performance/query-store-usage-scenarios?view=sql-server-ver17>

4. Best practices with Query Store

<https://learn.microsoft.com/en-us/sql/relational-databases/performance/best-practice-with-the-query-store?view=sql-server-ver17>

5. Query Store hints

<https://learn.microsoft.com/en-us/sql/relational-databases/performance/query-store-hints?view=sql-server-ver17>

6. Query Store hints best practices

<https://learn.microsoft.com/en-us/sql/relational-databases/performance/query-store-hints-best-practices?view=sql-server-ver17>

7. Parameter Sensitive Plan optimization

<https://learn.microsoft.com/en-us/sql/relational-databases/performance/parameter-sensitive-plan-optimization?view=sql-server-ver17>

8. Optional Parameter Plan Optimization

<https://learn.microsoft.com/en-us/sql/relational-databases/performance/optional-parameter-optimization?view=sql-server-ver17>

9. sp_query_store_force_plan

<https://learn.microsoft.com/en-us/sql/relational-databases/system-stored-procedures/sp-query-store-force-plan-transact-sql?view=sql-server-ver17>

10. SQL Server 2025 - What's new

<https://learn.microsoft.com/en-us/sql/sql-server/what-s-new-in-sql-server-2025?view=sql-server-ver17>

11. AdventureWorks sample databases

<https://learn.microsoft.com/en-us/sql/samples/adventureworks-install-configure?view=sql-server-ver17>

Appendix A — Query Store DBA Quick Reference

Goal	Primary object / command
Query Store health	sys.database_query_store_options
Find query text	sys.query_store_query_text + sys.query_store_query
Plan history / forcing state	sys.query_store_plan
Runtime metrics	sys.query_store_runtime_stats
Runtime intervals	sys.query_store_runtime_stats_interval
Query-level waits	sys.query_store_wait_stats
Query Store hints	sys.query_store_query_hints
PSP/OPPO variants	sys.query_store_query_variant
Force plan	sys.sp_query_store_force_plan
Unforce plan	sys.sp_query_store_unforce_plan
Apply hint	sys.sp_query_store_set_hints
Clear hint	sys.sp_query_store_clear_hints
Flush Query Store to disk	sys.sp_query_store_flush_db
Targeted procedure recompile	sys.sp_recompile

Appendix B — Core Concepts to Remember

1. Parameter sniffing is normal; parameter sensitivity is the performance problem.
2. Different compiled and runtime values alone do not prove a defect.
3. Prove skew, plan mismatch, and measurable regression before applying a fix.
4. Query Store preserves historical evidence that plan cache alone cannot provide.
5. Plan forcing is a stabilization tool and should include a rollback command.
6. Query Store hints are powerful because they do not require source-code changes, but they still require lifecycle review.
7. PSP should be evaluated first for eligible value-based parameter sensitivity on modern SQL Server.
8. RECOMPILE and disabled parameter sniffing prevent PSP from operating for the affected query.
9. OPPO is designed for optional NULL/non-NULL predicate patterns in SQL Server 2025 compatibility level 170.
10. Measure after every change using Query Store, actual execution plans, IO/time, and waits.